

Taller 03 v2 — Un solver multiagente con GraphRAG

MMIA 6013 IA Generativa y Agentes · USFQ · Integrante: Estefanía Gonzaga (trabajo individual)

Modelo (Partes 0–3): `zai-org/GLM-5.3-Flash`, leído de `http://172.28.230.10:12555/v1/models` (vLLM de la H200, réplica en el puerto 12559). **Embeddings:** `bge-m3:latest` en el Ollama de la H200 (:11434). **Juez (Parte 4.E):** `gemma3:27b` en el mismo Ollama. **Fechas de las corridas:** 2026-10-06 y 2026-10-07. Costo: 0 USD.

Integridad académica. Las tareas de práctica (A, B y C) se escribieron para este taller a partir de los papers del corpus del Taller 02. Las tareas reales (R1 y R2) son actividades de clase de la Semana 2 de esta materia; se convirtieron de notebook a PDF retirando mis soluciones. Si el solver se usa en una tarea de otra materia, se declara y se adjunta `corridas/<tarea>/traza.jsonl`.

Material

- **Base de conocimiento** (`corpus/`): los 16 papers del corpus del Taller 02 y las tres presentaciones de la Semana 2 (embeddings, bases vectoriales, fragmentación y recuperación). Son 1079 fragmentos de 300 palabras con 50 de solapamiento, cada uno con su cita (archivo y página). Entidades y embeddings se calculan una vez (`indexar_corpus.py`) y quedan en `.cache/`.
- **Tareas** (`enunciados/`):

Tarea	Origen	Pide	Entregable	Qué pone a prueba
A	Ng y Jordan (2001)	NB contra regresión logística en <code>load_digits</code> , curva de aprendizaje	<code>reporte.md</code> , máx. 1000 palabras	una parte que depende de otra; una conclusión del paper que los datos no confirman
B	Vaswani et al. (2017)	atención escalada, efecto de $\sqrt{d_k}$, codificación posicional	notebook ejecutado	cifras exactas con un generador consumido en orden
C	Blei et al. (2003)	LDA sobre 12 oraciones, perplejidad, pureza	<code>reporte.pdf</code> , máx. 2 páginas	datos dentro de tablas, límite de páginas, no descargar un corpus
R1	Real: actividad S2·MAR	kNN exacto, mini-IVF (recall contra <code>nprobe</code>), Qdrant	notebook ejecutado	código dado que usa la red (Qdrant en <code>localhost:6333</code>)
R2	Real: actividad S2·MIÉ	RAG mínimo: chunking, recuperación, abstención	notebook ejecutado	generación por APIs de red; modo inspección del propio notebook

Parte 0 — Tres errores que no dan error

Los tres scripts son propios y se aplican a la Tarea A. Usan las piezas reales del solver: el lector y las reglas del grafo (0.b), el sandbox y el revisor (0.c).

0.a — El solver que no ejecuta

Wed Oct 7 00:12:00 -05 2026

Medido ejecutando el código (`scikit-learn, random_state=7`):

```
nb_accuracy      0.8556
nb_f1_macro      0.8552
lr_accuracy      0.9711
lr_f1_macro      0.9711
nb_m20           0.3667
lr_m20           0.7044
nb_m50           0.6022
lr_m50           0.8511
nb_m100          0.7556
lr_m100          0.8956
nb_m200          0.8333
lr_m200          0.9333
```

```
nb_m400      0.8400
lr_m400      0.9667
nb_m1347     0.8556
lr_m1347     0.9711
```

LLM sin ejecutar: zai-org/GLM-5.3-Flash, 2026-10-07, 11598 tokens de salida,
fin=stop

30 cifras con decimales en el reporte:

```
0.844, 0.846, 0.969, 0.968, 0.489, 0.589, 0.100, 0.593, 0.744, 0.151, 0.669,
0.836, 0.167, 0.736, 0.909, 0.173, 0.796, 0.944, 0.148, 0.844, 0.969, 0.125, 0.589,
0.489 ...
```

Procedencia: 0 de 30 salen de una ejecución (no hubo ninguna).

¿Dice el reporte que las cifras son estimadas? no

Cifras que coinciden con alguna medida al redondear: 0 de 30

Las de m = 20 (la que decide si hay cruce), según el reporte: [' | 20 | 0.489 |
0.589 | 0.100 |'] · medido: NB 0.3667, LR 0.7044

Una corrida anterior del mismo script (parte0/salidas/0a_reporte_sin_ejecutar_corrida2.md) concluyó bien que no hay cruce, pero con cifras igual de inventadas (NB con m = 20: 0.311 contra 0.3667 medido) y con una nota que admite que «las cifras son los valores esperados».

1. **¿Por qué no basta con que las cifras estén cerca?** Una cifra cercana no es una medida: no se puede reproducir, no tiene error conocido y no dice qué código la produjo. El control de la Parte 4 lo muestra: este reporte aprueba la comprobación A07 (regresión logística 0.969, dentro de la tolerancia de 0.9711) sin haber ejecutado nada.
2. **¿Qué cifra inventada cambiaría una conclusión?** La exactitud con m = 20. El reporte da NB 0.489 contra LR 0.589. Si el modelo hubiera escrito NB > LR, como dice la teoría del paper, el reporte «confirmaría» el cruce de Ng y Jordan; lo medido es 0.3667 contra 0.7044, sin cruce.
3. **¿Qué debe comprobar un verificador sin conocer las respuestas?** La procedencia: que cada cifra aparezca en algo que escribió una ejecución (resultados.json, stdout, salidas del notebook). Aquí 0 de 30 aparecen en algo así. Es la REGLA 5 del solver.

0.b — El RAG que pierde la referencia

Pregunta del programador: «¿Con qué datos y con qué partición se calcula la curva de aprendizaje?»

- 1) RAG plano: los 2 fragmentos más parecidos (TF-IDF, coseno)

S3 «Parte 3 – La curva de aprendizaje» sim=0.319

S4 «Parte 4 – Discusión» sim=0.240

Hechos en el contexto: 0 de 3 []

- 2) Los mismos 2, más un salto por depende_de (regla, sin LLM)

S3 –depende_de→ S1, S2

S4 –depende_de→ (ninguna)

Hechos en el contexto: 3 de 3 ['conjunto de datos', 'proporción de prueba', 'prueba intocable']

Todas las aristas depende_de del enunciado:

S3 → S1 «...Con la partición de la Parte 1, entrena los dos modelos de la P...»

S3 → S2 «...ción de la Parte 1, entrena los dos modelos de la Parte 2 con 20, 50, 100, 200 y 400 ejemp...»

S5 → S2 «...oducción, Metodología, Resultados (la tabla de la Parte 2 y la figura de la Parte 3), Disc...»

S5 → S3 «...ultados (la tabla de la Parte 2 y la figura de la Parte 3), Discusión y Conclusiones. Máxi...»

RAG plano: 0 de 3 hechos. Con el salto: 3 de 3.

1. **¿Por qué es un fallo de la búsqueda y no del modelo?** El modelo nunca ve la Parte 1: la similitud elige la Parte 3 y la Parte 4 porque comparten las palabras de la pregunta, y los datos están en un texto que no se parece a ella. Con el contexto correcto, cualquier modelo copia la partición.
2. **¿Qué haría un LLM con el primer contexto?** Inventaría la partición (lo típico: 80/20 con `random_state=42`) o la preguntaría. Si la inventa, la curva usa otra división y sus cifras no coinciden con las de la Parte 2.
3. **¿Por qué la conexión se saca con una regla?** Porque la referencia «Parte 1» está escrita literalmente y una expresión regular la encuentra siempre, con costo cero y sin poder inventar una referencia que no existe. Pedírsela al LLM añade una fuente de error justo en la base del grafo.

0.c — Terminar sin error no es acertar

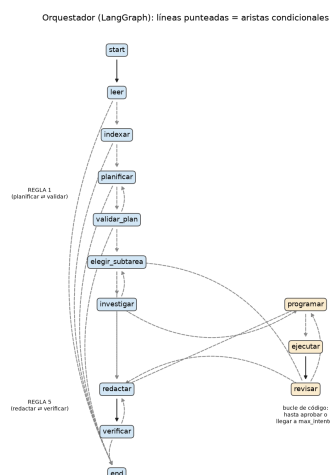
Revisión estática del sandbox (REGLA 3): permitido
 stdout del experimento: Exactitud: 1.000

- 1) Evaluador ingenuo: APROBADO (código de salida=0, resultados.json=sí)
- 2) Revisiones con código del revisor (REGLA 4):
 estática — se predice sobre lo mismo que se ajustó: ['X']
 plausible — métricas $\geq 0,999$ en un problema con ruido: ['accuracy=1.0']
 → RECHAZADO sin consultar al LLM. El programador recibe esto como corrección:
 — métricas implausibles ($\geq 0,999$) en un problema con ruido: ['accuracy=1.0'];
 ¿se evaluó sobre los datos de entrenamiento?
 — fuga de datos: se predice/evalúa sobre ['X'], la misma variable con que se ajustó

1. **¿Qué fuga no detectaría la revisión del código?** Un escalador ajustado con todos los datos antes de dividir (`StandardScaler().fit_transform(X)` y luego `train_test_split`): ningún `.predict` usa la variable de un `.fit` supervisado, y la exactitud resultante es plausible, solo un poco optimista.
2. **¿Por qué se puede dejar la decisión final a un LLM revisor si las revisiones con código van antes?** Porque el código descarta lo verificable y barato (código de salida, NaN, fuga evidente, cifra implausible) sin dejarse convencer; al LLM solo le llega lo que pasó esas barreras, y su papel es juzgar lo que el código no puede: si el script hace lo que pide el enunciado. En la primera medición de la Tarea C, el LLM rechazó una partición 75/25 que el enunciado no pedía y que el código no podía ver.

Parte 1 — El solver base

Arquitectura



El orquestador es un StateGraph de LangGraph 1.2.12 (solver/orquestador.py). El diagrama sale del grafo compilado (python diagrama_orquestador.py); las líneas punteadas son aristas condicionales, que deciden con código a qué nodo se pasa.

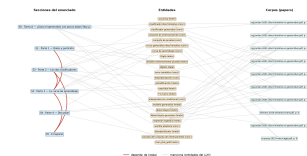
Rol	Módulo	Qué revisa su código
Lector	lector.py	PDF escaneado (< 200 caracteres útiles), tablas con find_tables, texto partido o pegado; el tamaño del cuerpo sin contar el código
Indexador (GraphRAG)	grafo.py, rag.py	secciones por tipografía; aristas depende_de por regex
Planificador	planificador.py	validar_plan
Investigador	investigador.py	sección literal + dependencias + preámbulo, cada fragmento con cita
Programador	programador.py	el filtro del sandbox
Ejecutor	sandbox.py (sin LLM)	entorno vacío, carpeta propia, tiempo máximo con killpg
Revisor	revisor.py	código de salida, resultados.json, NaN, figuras, fuga y plausibilidad
Redactor	redactor.py, formatos.py	secciones en orden, palabras, páginas, notebook ejecutado, origen de cada cifra

Las seis reglas, en el código (buscar REGLA n):

- 1. Plan validado con código** — planificador.validar_plan: ids únicos, dependencias que existen, sin ciclos (networkx.find_cycle), cada sección cubierta, formato de entrega válido. Si falla, validar_plan → planificar con la lista de problemas (máx. 3 rondas).
- 2. GraphRAG con base fija** — grafo.aristas_depends_de: «Parte N», «Tarea N», «P0», «la parte anterior» → arista depende_de por regex. Encima, entidades del LLM (dataset, método, métrica, concepto) del enunciado y del corpus, unidas por nombre normalizado. El investigador entrega la sección literal, sus dependencias (cierre transitivo), el preámbulo y los fragmentos del corpus que comparten entidades, cada uno con su cita.
- 3. Sandbox** — sandbox.revisar_codigo (AST: sin red, sin procesos, sin borrar, sin eval, sin rutas fuera de la carpeta), sandbox.entorno_limpio (el entorno se construye desde cero: ninguna variable se hereda) y sandbox.ejecutar (carpeta propia, start_new_session y os.killpg al vencer el tiempo). El notebook se ejecuta con el mismo entorno.
- 4. Resultados en archivo fijo y revisión con código primero** — cada script escribe resultados.json; revisor.revisiones_con_codigo corre antes que el LLM y, si encuentra algo, el LLM no se consulta.
- 5. Origen de cada cifra** — procedencia.cifras_sin_origen, dentro de redactor.validar_entregable: cada cifra del entregable debe aparecer en una ejecución o en el enunciado (salvo los enteros de 0 a 10, numeración). Si no, verificar → redactar con la lista.
- 6. Traza siempre** — traza.Traza: una línea JSON por evento, escrita al momento (también si el proceso muere). Por llamada: agente, modelo, tokens de entrada y salida, latencia, error; por ejecución: código de salida, duración, archivos creados. Los mensajes completos van a traza_llm/.

Contrato. Solver().solve(pdf, salida) devuelve status, entregables, subtareas, usage (con el detalle por agente), model y trace; Solver().run(ruta) devuelve answer, trace, status, model y usage (Taller 4).

El grafo de la Tarea A



Las cuatro aristas `depende_de` (en rojo) salen de la regla: $S3 \rightarrow S1$ y $S3 \rightarrow S2$ («Con la partición de la Parte 1, entrena los dos modelos de la Parte 2»), $S5 \rightarrow S2$ y $S5 \rightarrow S3$ («la tabla de la Parte 2 y la figura de la Parte 3»). Las entidades unen el enunciado con las páginas del paper de Ng y Jordan.

El plan de la Tarea A (`corridas/completo/tarea-A/plan.json`)

Subtarea	Tipo	Secciones	Depende de	Figura
T1 Datos y partición única	cálculo	S1	—	no
T2 GaussianNB contra regresión logística	cálculo	S2	T1	no
T3 Curva de aprendizaje con 20, 50, 100, 200, 400 y todos	cálculo	S3	T1, T2	sí
T4 Redacción del reporte y discusión	redacción	S4, S5	T2, T3	no

Entrega: `md`, `reporte.md`, secciones Introducción, Metodología, Resultados, Discusión y Conclusiones, 1000 palabras. El plan pasó la validación en la primera ronda.

Una corrida en la que se rechazaron scripts: tarea real R1 (`corridas/completo/tarea-R1/traza.jsonl`)

Evento	Subtarea · intento	Qué pasó
21	T2 · 1	Rechazado por el sandbox: «error de sintaxis en la línea 1» (el modelo devolvió texto sin bloque de código)
24–26	T2 · 2	Ejecución con código 0; aprobado por el LLM
30–31	T3 · 1	El script copia el código dado: <code>QdrantClient(url="http://localhost:6333")</code> . FRENO 4: nadie aprueba la red → rechazado
34–36	T3 · 2	Qdrant en modo embebido (<code>:memory:</code>); aprobado por el LLM

Parte 2 — Evaluación

2.a — El golden set (`golden_tareas.json`)

43 comprobaciones: 9 en cada tarea de práctica y 8 en cada tarea real. Cada tarea tiene comprobaciones de forma (archivo, secciones en orden, palabras o páginas, notebook ejecutado), al menos dos de tipo cifra con su `verdad_py` y una de procedencia. **Ninguna verdad se escribe a mano:** `verdad_py` llama a `verdades.py`, que reproduce cada enunciado con sus parámetros.

- **Notebook:** B, R1, R2. **PDF:** C. **Trampas:** A (partición única, escalador solo con el entrenamiento y un cruce del paper que los datos no muestran); B (un solo generador consumido en orden fijo); C (no descargar un corpus); R1 (Qdrant por URL en el código dado); R2 (generación por APIs de red). Las tres últimas usan el tipo nuevo `codigo_sin` sobre el código que se ejecutó.
- **Tareas reales.** `enunciados/reales/convertir_actividades.py` convierte los notebooks de clase en PDF: conserva el Markdown y el código que da el profesor y, de las celdas de ejercicio y de reflexión, solo las líneas de comentario que dicen qué hacer. La verdad de R1 se validó contra las salidas guardadas del notebook (celdas de 1562, puntajes de Qdrant -0.102 y -0.112).
- **Lo que enseñó medir el golden set.**
 - *Una verdad no reproducible.* El `recall@10` de R1 da 0.916 o 0.926 con `nprobe = 1` en corridas idénticas: KMeans multihilo cambia el orden de las sumas. Solo lleva tolerancia 0.015; las comprobaciones exactas van sobre cifras deterministas.
 - *Un enunciado ambiguo* (A, primera versión): «NB y regresión logística con los atributos estandarizados» se leyó como estandarizar ambos; NB dio 0.8089 en lugar de 0.8556. Se corrigió el enunciado.
 - *Una verdad sobre texto que el solver no ve* (R204): la pregunta corta del ejercicio 2.1 solo estaba en mi solución, que la conversión retira. Se cambió por la pregunta del código dado.

- Una comprobación de trampa frágil (`codigo_sin`): contaba intentos rechazados que nunca corrieron, comentarios y docstrings que solo mencionan una URL. Se afinó tres veces; lo robusto sería reutilizar el AST, como el sandbox.

- **Evaluador:** `evaluar_solver.py` es propio y no importa nada del solver. Los artefactos del solver (grafo, traza, plan, intentos) no cuentan como entrega ni como evidencia; `--reanudar` permite seguir una evaluación cortada (se usó tras una caída de la VPN).

2.b — Solver completo contra la versión sin grafo

Mismo golden set y mismo modelo. En la versión **sin grafo**, el investigador solo recibe los $k = 4$ fragmentos más parecidos a la subtask (secciones y corpus juntos, con bge-m3), como en la Parte 0.b, y el indexador no extrae entidades. Las tablas salen de `resultados/*.csv` con `python tablas.py` (Checks = comprobaciones aprobadas; Proced. = fracción de cifras respaldadas por una ejecución; Fallidas = subtasks fallidas; s = duración en segundos).

Por tarea y versión

Tarea	Versión	Status	Checks	Proced.	Fallidas	Intentos	Tok. entrada	Tok. salida	s
A	completo	completado	9/9	1.00	0/4	3	28517	41918	227.22
A	sin_grafo	completado	9/9	1.00	0/4	4	25777	67200	393.28
B	completo	completado	9/9	1.00	0/5	3	51065	58362	320.09
B	sin_grafo	completado	9/9	1.00	0/4	3	37793	52918	304.11
C	completo	completado	9/9	1.00	0/4	3	53479	89684	555.39
C	sin_grafo	parcial	8/9	1.00	1/4	6	59387	155599	946.35
R1	completo	completado	7/8	0.81	0/4	5	76215	99936	1629.61
R1	sin_grafo	parcial	6/8	0.82	1/4	5	60433	97396	629.95
R2	completo	parcial	7/8	1.00	0/4	3	65266	378002	2200.04
R2	sin_grafo	parcial	4/8	0.83	1/4	5	70567	347693	2070.87

Totales por versión

Versión	Aprobadas	Total	Subtasks fallidas	Intentos	Tokens entrada	Tokens salida	Duración (s)
completo	41	43	0	17	274542	667902	4932.3
sin_grafo	36	43	3	23	253957	720806	4344.6

Tokens por agente (suma de las tareas)

Agente	llamadas (completo)	entrada (completo)	salida (completo)	llamadas (sin_grafo)	entrada (sin_grafo)	salida (sin_grafo)
programador	24	88190	490320	30	100405	575352
revisor	15	103333	76072	14	80679	42723
redactor	6	55730	60468	8	60977	87057
planificador	5	11896	15211	5	11896	15674
indexador	28	15393	25831	0	0	0

Tokens por tarea y agente (entrada / salida)

Tarea	Agente	completo	sin_grafo
A	programador	7000 / 23597	7375 / 49847
A	revisor	13381 / 5193	12473 / 4144
A	redactor	4662 / 6064	4583 / 11450
A	planificador	1346 / 2036	1346 / 1759
A	indexador	2128 / 5028	0 / 0
B	programador	8229 / 15182	5642 / 13149

B	revisor	17542 / 13119	13543 / 11571
B	redactor	20822 / 23637	16769 / 25112
B	planificador	1839 / 2052	1839 / 3086
B	indexador	2633 / 4372	0 / 0
C	programador	10631 / 34942	23135 / 130790
C	revisor	29290 / 38401	18215 / 11565
C	redactor	8052 / 10644	15682 / 11340
C	planificador	2355 / 1795	2355 / 1904
C	indexador	3151 / 3902	0 / 0
R1	programador	23291 / 64972	18697 / 58889
R1	revisor	24902 / 7180	21632 / 8470
R1	redactor	22194 / 20123	17473 / 26837
R1	planificador	2631 / 2286	2631 / 3200
R1	indexador	3197 / 5375	0 / 0
R2	programador	39039 / 351627	45556 / 322677
R2	revisor	18218 / 12179	14816 / 6973
R2	redactor	0 / 0	6470 / 12318
R2	planificador	3725 / 7042	3725 / 5725
R2	indexador	4284 / 7154	0 / 0

Comprobaciones: completo frente a sin_grafo

Tarea	Check	Tipo	Completo	Sin grafo	Detalle (completo)
A	A01	archivo	✓	✓	1 archivo(s) con reporte.md
A	A02	archivo	✓	✓	1 archivo(s) con **/*.png
A	A03	secciones	✓	✓	todas, en orden
A	A04	palabras_max	✓	✓	792 palabras
A	A05	cifra_presente	✓	✓	verdad=1797.0000 ±0; hallada
A	A06	cifra	✓	✓	verdad=0.8556 ±0.006; hallada
A	A07	cifra	✓	✓	verdad=0.9711 ±0.006; hallada
A	A08	cifra_presente	✓	✓	verdad=0.3667 ±0.0006; hallada
A	A09	procedencia	✓	✓	37/37 cifras respaldadas (1.00)
B	B01	archivo	✓	✓	1 archivo(s) con *.ipynb
B	B02	ipynb_ejecutado	✓	✓	4 celdas, 0 sin ejecutar, 0 con error
B	B03	cifra_presente	✓	✓	verdad=0.4981 ±0.00051; hallada
B	B04	cifra_presente	✓	✓	verdad=0.8639 ±0.00051; hallada
B	B05	cifra_presente	✓	✓	verdad=0.3537 ±0.00051; hallada
B	B06	cifra_presente	✓	✓	verdad=3.0363 ±0.00051; hallada
B	B07	cifra_presente	✓	✓	verdad=0.7108 ±0.00051; hallada
B	B08	archivo	✓	✓	2 archivo(s) con **/*.png
B	B09	procedencia	✓	✓	60/60 cifras respaldadas (1.00)
C	C01	archivo	✓	✓	1 archivo(s) con reporte.pdf
C	C02	paginas_max	✓	✓	2 páginas
C	C03	secciones	✓	✓	todas, en orden
C	C04	cifra	✓	✓	verdad=137.7877 ±0.06; hallada
C	C05	cifra_presente	✓	✗	verdad=156.4198 ±0.06; hallada
C	C06	cifra	✓	✓	verdad=0.5000 ±0.006; hallada
C	C07	cifra_presente	✓	✓	verdad=71.0000 ±0; hallada
C	C08	codigo_sin	✓	✓	ninguno

C	C09	procedencia	✓	✓	22/22 cifras respaldadas (1.00)
R1	R101	archivo	✓	✓	1 archivo(s) con *.ipynb
R1	R102	ipynb_ejecutado	✓	✓	4 celdas, 0 sin ejecutar, 0 con error
R1	R103	cifra_presente	✓	✓	verdad=1562.5000 ±0.6; hallada
R1	R104	cifra	✓	✓	verdad=0.9920 ±0.015; hallada
R1	R105	cifra_presente	✓	✗	verdad=0.1018 ±0.0006; hallada
R1	R106	archivo	✓	✓	2 archivo(s) con **/*.png
R1	R107	codigo_sin	✓	✓	ninguno
R1	R108	procedencia	✗	✗	56/69 cifras respaldadas (0.81)
R2	R201	archivo	✓	✓	1 archivo(s) con *.ipynb
R2	R202	ipynb_ejecutado	✓	✓	3 celdas, 0 sin ejecutar, 0 con error
R2	R203	cifra	✓	✗	verdad=6.0000 ±0; hallada
R2	R204	cifra_presente	✓	✗	verdad=0.4986 ±0.0006; hallada
R2	R205	cifra_presente	✓	✗	verdad=0.3985 ±0.0006; hallada
R2	R206	contiene	✗	✓	faltan ['El corpus no contiene información suficiente']
R2	R207	codigo_sin	✓	✓	ninguno
R2	R208	procedencia	✓	✗	sin cifras con dos o más decimales

Lectura. El grafo ganó 5 comprobaciones (41 contra 36), 3 subtareas fallidas menos y 6 intentos de código menos, con un 3 % menos de tokens (942 444 contra 974 763); tardó más (4932 s contra 4345 s), sobre todo por R1 y R2. Las diferencias están donde el contexto importa: en C, sin el preámbulo con la tabla de los 12 documentos, T2 no tuvo datos; en R1, la puntuación de Qdrant (R105) no llegó al entregable porque T3 falló; en R2, la versión sin grafo no reportó el número de fragmentos ni los puntajes (R203–R205). Ambas versiones fallaron la procedencia de R1 por un defecto del ensamblado del notebook (2.c).

2.c — Los tres peores casos

Evidencia en `resultados/peores_casos_completo.md` y `peores_casos_sin_grafo.md` (sacada de la traza con `peores_casos.py`). Cada corrección se midió después con el mismo golden set:

Tarea	Versión	Etapas	Status	Checks	Proced.	Fallidas	Intentos	Tokens	s
C	completo	v1: medición inicial	completado	9/9	1.00	0/4	5	169635	615.6
C	completo	revisor: fuga solo en ajustes supervisados	completado	9/9	1.00	0/4	4	185077	1241.3
C	sin_grafo	v1: medición inicial	parcial	7/9	0.88	1/4	4	408976	2158.3
C	sin_grafo	revisor: fuga solo en ajustes supervisados	parcial	7/9	1.00	1/4	4	411970	3230.0
R1	completo	medición oficial	completado	7/8	0.81	0/4	5	176151	1629.61
R1	completo	notebook sobre una copia (trabajo_nb/)	parcial	7/8	1.00	1/4	5	208937	1076.79
R1	completo	revisor sabe que no hay red; plausibilidad solo en clasificadores	completado	8/8	1.00	0/4	5	250641	1250.67
R1	sin_grafo	medición oficial	parcial	6/8	0.82	1/4	5	157829	629.95
R1	sin_grafo	notebook sobre una copia (trabajo_nb/)	completado	8/8	1.00	0/4	5	319346	1887.87
R1	sin_grafo	revisor sabe que no hay red; plausibilidad solo en clasificadores	completado	8/8	1.00	0/4	5	166275	894.94

R2	completo	medición oficial	parcial	7/8	1.00	0/4	3	443268	2200.04
R2	completo	presupuesto en los reintentos por max_tokens	parcial	6/8	0.93	0/4	3	406694	2084.76
R2	sin_grafo	medición oficial	parcial	4/8	0.83	1/4	5	418260	2070.87
R2	sin_grafo	presupuesto en los reintentos por max_tokens	parcial	5/8	1.00	0/4	3	351772	1706.35

- R206 — la abstención no llega al entregable (completo, R2). Agentes: programador y freno de presupuesto.** Recibió para T3 (generación) el código dado, que llama por red a tres APIs. Produjo un primer intento sin resultados.json (rechazado por el código) y después razonó hasta agotar max_tokens seis veces. El código decidió: el reintento de 65 536 tokens se autorizó con la estimación típica (unos 5000), así que una sola llamada pasó el límite (443 268 usados contra 390 000) y se comió la reserva del redactor; el redactor tampoco pudo llamar al LLM y el solver publicó el entregable mínimo escrito por código, sin la frase de abstención. Corrección: un reintento se autoriza con el nuevo max_tokens. Medido: el freno salta antes del reintento («usados 383 265 + estimados 71 897 > límite 390 000») y el redactor escribe el entregable.
- R108 — procedencia 0.81 (completo y sin grafo, R1). Agente: redactor, en el ensamblado del notebook.** R1 mide latencias, que cambian en cada ejecución. El redactor copió las del script aprobado, pero el notebook volvía a correr cada script en su carpeta y sobrescribía el resultados.json de donde salían. La REGLA 5 pasó porque comparaba contra los valores en memoria; el evaluador, contra los archivos. Corrección: el notebook corre sobre una copia (trabajo_nb/). Medido: procedencia 1.00 en las dos versiones.
- R105 — el puntaje de Qdrant no llega al entregable (sin grafo, R1). Agentes: revisor y sandbox, en conflicto.** T3 copió el código dado (localhost:6333) y el freno de red lo bloqueó (bien). El programador quitó la conexión y el revisor LLM lo rechazó porque «el enunciado exige intentar conexión a localhost:6333»: nadie le decía que la red está prohibida. Tercer intento con la URL otra vez, bloqueado; T3 fallida. En la re-medición apareció un segundo falso positivo: la revisión de plausibilidad rechazaba un recall@10 = 1.0, legítimo en un índice ANN. Corrección: el prompt del revisor dice que el sandbox prohíbe la red y que la alternativa local del enunciado es la correcta; la plausibilidad solo mira clasificadores. Medido: R1 8/8 en las dos versiones.

De la primera medición (v1, tres tareas) queda otra corrección medida: la revisión de fugas rechazaba `lda.fit(X) + lda.score(X)` en la Tarea C, que en un modelo no supervisado es lo que se pide; ahora solo cuenta ajustes supervisados (`corridas/v1/`).

Parte 3 — Los cuatro frenos

Los cuatro se hicieron saltar con un modelo de guion (`frenos.py`, sin gastar tokens; trazas en `corridas/frenos/<escenario>/traza.jsonl`; salida en `resultados/frenos_salida.txt`), y tres saltaron solos en las corridas oficiales.

Freno	Demostración (<code>frenos.py</code>)	Evento en la traza	En las corridas oficiales
Presupuesto, revisado antes de cada llamada, con reserva para el redactor	presupuesto: total 20 000, reserva 7000. T1 se completa; antes de programar T2, «usados 9080 + estimados 7534 > límite 13000». Se entrega parcial	freno_presupuesto	3 veces (R2 en las dos versiones)

Máximo de intentos y detector de repetición	intentos: T2 entrega un árbol con fuga; el intento 1 se rechaza; los intentos 2 y 3 son idénticos (misma huella AST) y no se ejecutan ; T2 queda fallida y T3 se completa	freno_repeticion x2, freno_intentos	3 veces (C, R1 y R2 sin grafo)
Tiempo máximo que mata todos los procesos	timeout: while True muerto a los 3 s; T1 se reescribe. Prueba directa: un script que lanza un hijo; tras el tiempo, el hijo ya no existe (hijo_sigue_vivo: false)	freno_timeout	no hizo falta
Confirmación humana antes de usar la red	red: T1 llama a fetch_openml; la persona responde «no»; el programador reescribe con load_digits	freno_red (decisión, quién)	3 veces (Qdrant por URL en R1)

Además, el reintento por `finish_reason=length` con contenido vacío saltó 14 veces en las corridas oficiales (`reintento_llm`).

Parte 4 — Mejora E: un juez de otra familia

`juez_gemma.py: gemma3:27b` (otra familia que GLM) recibe el enunciado, el entregable y la lista de archivos, y decide criterio por criterio (la `descripcion` de cada comprobación del golden set) si se cumple, sin ver la verdad calculada. Se compara con las comprobaciones de código; el grupo `control_sin_ejecutar` es el reporte de la Parte 0.a entregado como si fuera la Tarea A.

grupo	n	acuerdo	kappa	aprueba evaluador	aprueba juez	juez aprueba y codigo rechaza	juez rechaza y codigo aprueba
completo	43	0.953	0.0	41	43	2	0
sin_grafo	43	0.907	0.659	36	36	2	2
control_sin_ejecutar	9	0.556	0.0	5	9	4	0
total	95	0.895	0.447	82	88	8	2

Sobre las entregas del solver completo, el juez lo aprueba todo: 43 de 43 criterios, contra 41 de las comprobaciones de código. Coincide en el 95.3 % de los criterios, pero el kappa es 0.0: como el juez nunca rechaza nada, su acuerdo es el que daría el azar. Los dos criterios en que discrepa son los que el código rechaza: la procedencia de R1 (56 de 69 cifras respaldadas) y la abstención ausente en R2. En la versión sin grafo, con más fallas visibles en el texto («no disponible»), el kappa sube a 0.659.

El control adversarial lo confirma. Se le dio como entrega de la Tarea A el reporte de la Parte 0.a: bien redactado y con 30 cifras inventadas. Las comprobaciones de código aprueban 5 de 9 (fallan la figura, dos cifras y la procedencia, 0/30); el juez aprueba 9 de 9: acuerdo 0.556 y kappa 0.0. Sus razones muestran el mecanismo: acepta cada cifra inventada porque «es plausible» (NB 0.844 con el entrenamiento completo; lo medido es 0.8556) y da por buena la procedencia porque «el reporte incluye un bloque de código que parece generar todas las cifras». La figura la aprueba porque «el reporte menciona que se guarda».

Conclusión. En total (95 criterios), acuerdo 0.895 y kappa 0.447, y todo el desacuerdo es en la misma dirección: el juez aprueba lo que el código rechaza (8 casos) mucho más que lo contrario (2). Un juez LLM, incluso de otra familia, no sustituye a las comprobaciones con código: no puede ejecutar nada y juzga si el texto *parece* correcto. Sirve como segunda opinión sobre lo que el código no mide (la calidad de la discusión), siempre después de las comprobaciones de código, que es el orden de la REGLA 4.

Parte 5 — Preguntas

1. ¿Quién decide que la tarea está resuelta? Ninguno de los agentes con LLM. El planificador propone subtareas y el revisor aprueba scripts, pero el `status` final lo calcula `Solver._status` (`solver/solver.py`): «completado» solo si todas las subtareas de cálculo quedaron aprobadas, el entregable pasó `validar_entregable` (secciones, extensión, notebook ejecutado y REGLA 5) y no se agotó el presupuesto. Si algo de eso falla, «parcial»; sin entregable, «fallido». Y fuera del solver, la calificación la decide el evaluador, que es código y no comparte nada con el solver. Un LLM puede

equivocarse al aprobar un script, pero no puede declarar resuelta una tarea cuyas cifras no salieron de una ejecución.

2. ¿Qué agente gasta más tokens y qué ganó el grafo? El programador: 88 190 de entrada y 490 320 de salida en la versión completa, el 61 % de los tokens (`tablas.md`). Gasta más porque el modelo razona antes de escribir cada script y esos tokens se cobran como salida; cuando la tarea lo enreda (la generación por red de R2) razona hasta agotar `max_tokens` varias veces. El revisor es el segundo, porque recibe código, salida y contexto completos. El grafo ganó 5 comprobaciones (41 contra 36), 3 subtareas fallidas menos y 6 intentos de código menos, con 32 319 tokens menos (−3 %); a cambio tardó 588 s más, por las entidades del índice y un contexto más largo que el revisor lee entero.

3. Uso honesto y riesgos. Usarlo de forma honesta exige declararlo, adjuntar la traza (qué recibió cada agente, qué código se ejecutó, qué rechazó el revisor) y los scripts, separar lo que escribió el sistema de lo que revisó o cambió la persona, y no presentar como propias las decisiones del planificador ni las cifras sin verificar su procedencia. Si el sandbox fallara, el código generado correría con los permisos del usuario: podría borrar archivos, leer credenciales del entorno o del disco y enviarlas por red, descargar y ejecutar código de terceros, o dejar procesos vivos consumiendo recursos. Por eso el entorno va vacío, la carpeta es desechable, la red pide una persona y todo proceso muere con su grupo; un aislamiento completo pediría además un contenedor sin red.

Reproducibilidad

`README.md` tiene todos los comandos. Versiones exactas en `requirements.txt` (Python 3.14.7). Corridas oficiales en `corridas/completo/` y `corridas/sin_grafo/`; re-mediciones en `corridas/fix/` y `corridas/fix2/`; primera medición (tres tareas) en `corridas/v1/`; control del juez en `corridas/control_sin_ejecutar/`; frenos en `corridas/frenos/`. Cada tarea tiene `traza.jsonl`, `traza_llm/`, `plan.json`, `grafo.png`, `trabajo/` y su entregable. CSV en `resultados/` (y `resultados/fix*`, `resultados/v1`). Ninguna clave aparece en el código, las trazas ni los scripts generados: la H200 no la pide y el sandbox no hereda el entorno.